

Presentation: Meyer's Theorem

Definition: We define the class $\text{SIZE}(n^c)$ as problems that can be solved by a boolean circuit with $O(n^c)$ gates.

Definition: P/poly is $\bigcup \text{SIZE}(n^c)$

Definition: EXP is $\bigcup \text{DTIME}(2^{n^c})$.

Theorem (Meyer): If $\text{EXP} \subseteq \text{P/poly}$, then $\text{EXP} = \Sigma_2^P$ (from the polynomial hierarchy).

Proof: First, for a language $L \in \text{EXP}$, there is an oblivious Turing Machine M_L that decides L in $2^{p(n)}$ steps for some polynomial p . Since the Turing Machine is oblivious, we know for any step i the last step i'_j such that the head of tape j visited the same cell. Moreover, the transition function is constant.

We can create a "snapshot" $z_i := (q_i, r_{i1}, \dots, r_{ik})$. We can tell if a snapshot z is part of this TM's runtime in exponential time just by running M .

By hypothesis, $\text{EXP} \subseteq \text{P/poly}$, so there is a polynomial circuit family $C_{x(i)} = z_i$. There is a machine $T(x, C_x, i, i'_1, \dots)$ that outputs 1 if $C_x(i) = C_x(i'_1) \dots = C_x(i'_j)$. Adding the appropriate quantifiers, we get

$$\exists C_i, \forall i, i'_1, \dots, i'_j, T(x, C_x, i, i'_1, \dots) = 1$$

which is Σ_2^P and decides L .

Lecture

SAT and TISP

Theorem: $\text{SAT} \notin \text{TISP}(n^{1.1}, n^{0.1})$

Proof: We prove three claims to use later:

- *Claim 1:* If $\text{SAT} \in \text{TISP}(n^{1.1}, n^{0.1})$, then $\text{NTIME}(n) \subseteq \text{TISP}(n^{1.2}, n^{0.2})$. This follows from reducing $\text{NTIME}(n)$ to a SAT problem, and is omitted.
- *Claim 2:* $\text{TISP}(n^{12}, n^2) \subseteq \sum_2 \text{TIME}(n^8)$. *Proof:* Let $L \in \text{TISP}(n^{12}, n^2)$, and M compute L in $O(n^{12})$ time and $O(n^2)$ space. Fix x an input of M . We can list the $O(n^{12})$ configurations of M as $C_0, \dots, C_k$. Each configuration takes $O(n^2)$ space. We can sample configurations $C_0, C_{n^6}, C_{2n^6}, \dots$ to get a $O(n^8)$ space witness of the C_i s. To check if $x \in L$, we rephrase the question as:
 $\exists C_0, C_{n^6}, C_{2n^6}, \dots, \forall i \in [O(n^6)], M'(x, i, C_0, \dots)$ where M' runs M from one C_i configuration to the next C_{i+1} (in $O(n^6)$ time) and checks it matches the witness (and that the last witness is an accepting state). Reading the witness takes $O(n^8)$, so this reformulation shows L is in $\sum_2 \text{TIME}(n^8)$.
- *Claim 3:* If $\text{NTIME}(n) \subseteq \text{DTIME}(n^{1.2})$, then $\sum_2 \text{TIME}(n^8) \subseteq \text{NTIME}(n^{9.6})$.

Let $L \in \sum_2 \text{TIME}(n^8)$. We want to show that $L \in \text{NTIME}(n^{9.6})$.

By hypothesis and some padding, we know that $\text{NTIME}(n^8) \subseteq \text{DTIME}(n^{9.6})$. This also implies $\text{coNTIME}(n^8) \subseteq \text{DTIME}(n^{9.6})$ since DTIME is equal to its complement.

Define a language $L^*(x, u) := 1$ if $\forall v, M(x, u, v) = 1$, and 0 otherwise. L^* is in $\text{coNTIME}(n^8)$ by construction, and so is in $\text{DTIME}(n^{9.6})$. Thus, there is machine M^* that computes L^* in $O(n^{9.6})$. Then $L(x) = 1$ iff $\exists u, M^*(x, u)$. This shows $L \in \text{NTIME}(n^{9.6})$.

With these three claims, then we can say:

$$\text{NTIME}(n^{10}) \subseteq \text{TISP}(n^{12}, n^2) \subseteq \sum_2 \text{TIME}(n^8) \subseteq \text{NTIME}(n^{9.6})$$

which contradicts the non-uniform time hierarchy theorem.

Oracle Classes

We briefly discussed oracle classes in previous lectures. As a reminder, we give an example:

- NP^{SAT} consist of L such that there exists TMs M with $L(x) = 1 \Leftrightarrow \exists w, M^{\text{SAT}}(x, w)$. Recall that M^{SAT} is M that is allowed to query an “oracle” to solve SAT in constant time.

Theorem: $\text{NP}^{\sum_{i=1}^p \text{SAT}} = \sum_i^p$, where $\sum_i^p \text{SAT}$ is a $\sum_{i=1}^p$ -complete problem (typically boolean SAT with $i - 1$ quantifiers starting with $\exists$). In particular, $\text{NP}^{\text{SAT}} = \sum_2^p$.

The proof is omitted, but should not be difficult.

Non-Uniform Complexity and Hierarchy Theorem

Definition: $\text{DTIME}(T(n))/a(n)$ is class of languages L such that there is a TM M such that $M(x; \text{adv}) = L(x)$ in time $O(T(n))$ and $|\text{adv}| = O(a(n))$. The advice can only depend on $|n|$, not its content. Such a turing machine is called a **non-uniform** Turing Machine.

This definition is equivalent the circuit computation model.

Definition: P/poly is $\bigcup_{c,d \geq 1} \text{DTIME}(n^c)/n^d$.

Consider the language UHALT that contains the strings 1^n (ie. n ones), where the n th Turing machine halts. This language is undecidable for uniform Turing Machines, but it is in P/poly .